Grid Layout

Выполнили: Илютикова Екатерина, Воронкина Варвара, Червякова Амалия

ИБ-31БО

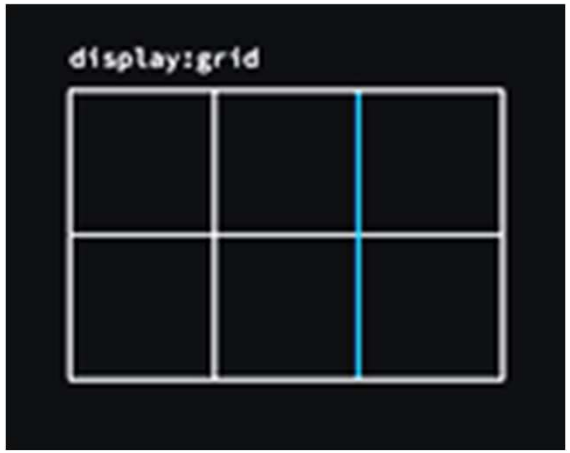
Что такое гриды?

- CSS Grid Layout (спецификация) или просто *гриды* — это удобная технология для раскладки элементов на веб-страницах. Гриды дают возможность работать одновременно с двумя: горизонталью и вертикалью.

- Принцип работы гридов чем-то похож на таблицы. Вместо работы только с рядами или только с колонками с помощью гридов можно работать с так называемыми *грид-ячейками*, позиционируя элементы по вертикали и горизонтали одновременно.

Основные термины

- Грид-контейнер: родительский элемент, к которому применяется свойство display: grid.
- Грид-элемент: дочерний элемент, прямой потомок грид-контейнера. Подчиняется правилам раскладки гридов.
- Грид-линия: разделительная линия, формирующая структуру грида. Может быть как вертикальной (грид-линия колонки), так и горизонтальной (грид-линия ряда). Располагается по обе стороны от колонки или ряда. Используется для привязки грид-элементов.



Основные термины

- **Грид-ячейка**: пространство между соседними грид-линиями. Единица грид-сетки.
- **Грид-полоса**: пространство между двумя соседними грид-линиями. Может быть проще думать о грид-полосе как о ряде или колонке.
- **Грид-область**: область, ограниченная четырьмя грид-линиями. Может состоять из любого количества ячеек как по горизонтали, так и по вертикали.

Свойства грид-контейнера: display

- Display: grid
- Если элементу задано свойство display со значением grid, то такой элемент становится грид-контейнером. Дочерние элементы этого контейнера начинают подчиняться правилам грид-раскладки. Снаружи грид-контейнер ведёт себя как блок.
- Display: inline-grid
- Практически аналогичное display: grid значению — за тем исключением, что снаружи грид-контейнер, созданный с помощью inline-grid, будет вести себя как строчный элемент.

```
.container {
  display: grid;
}
```

```
.container {
  display: inline-grid;
}
```

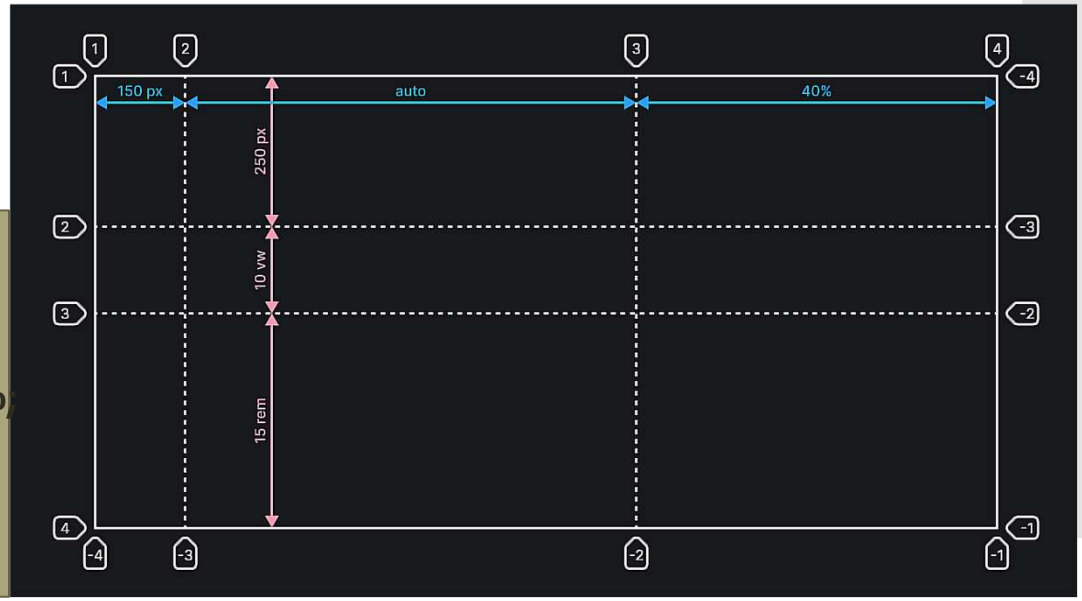
Свойства грид-контейнера: display

Создание сетки

- **grid-template-columns, grid-template-rows**

- Свойства, задающие размеры и количество колонок или рядов грид-раскладки соответственно.
- После двоеточия указываются размеры каждой колонки или ряда, в том формате, который удобен с той единицей измерения, которая нужна вам.

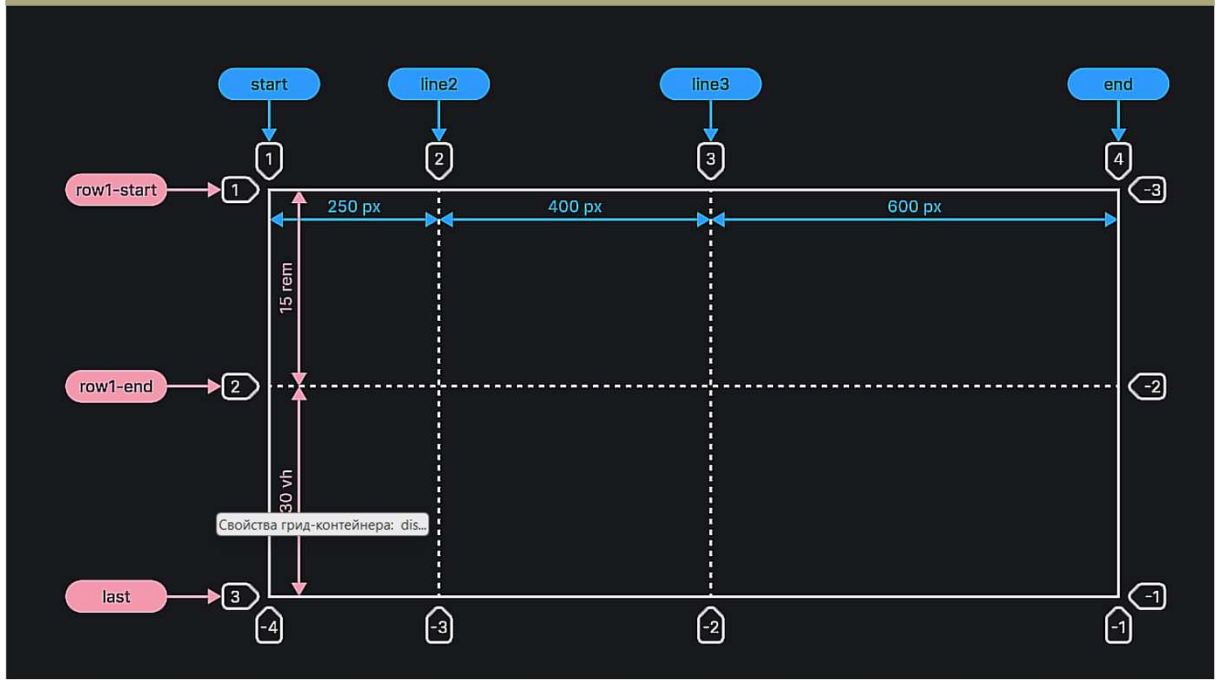
```
.container {
  display: grid;
  /* Будет создано 3 колонки */
  grid-template-columns: 150px auto 40%;
  /* Будет создано 3 ряда */
  grid-template-rows: 250px 10vw 15rem;
}
```



Свойства грид-контейнера: display

- Можно явно именовать грид-линии, используя для этого квадратные скобки. Это позволяет обозначать начало и конец областей, избегать случайного изменения структуры сетки, а так же для упрощения адаптивности:

```
.container {
  display: grid;
  grid-template-columns: [start] 250px [line2] 400px [line3] 600px [end];
  grid-template-rows: [row1-start] 15rem [row1-end] 30vh [last];
}
```



Свойства грид-контейнера: display

Отзывчивые сетки

- Если нужны одинаковые колонки или ряды, то можно воспользоваться функцией repeat().
- Будет создано 3 колонки по 250 пикселей:

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 250px);
}
```

Свойства грид-контейнера: display

Единицы измерения

- С появлением гридов у нас появилась и новая единица измерения: fr
- fr (от *fraction* — доля, часть) отвечает за свободное пространство внутри грид-контейнера.
- Например, этот код создаст три колонки, каждая из которых будет занимать 1/3 ширины родителя:

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

- Что аналогично записи:

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

Свойства грид-контейнера

Единицы измерения

- **Auto** — значение, при котором **размер ряда или колонки подстраивается** под размеры элементов, так, чтобы уместился самый большой из них. Если в контейнере есть свободное место, то размер может растянуться до конца контейнера.
- Концепция автоматического размещения связана с различием между явной (explicit) и неявной (implicit) сеткой:
- **Явная сетка** — определяется через свойства grid-template-rows и grid-template-columns, точно задаётся количество и размеры строк и столбцов.
- **Неявная сетка** — создаётся автоматически, когда элементы выходят за границы явной сетки.

Свойства грид-контейнера

- **Принцип работы**
- Элементы, которые не размещены явно, автоматически помещаются в незанятое пространство в контейнере-сетке с помощью алгоритма автоматического размещения. Некоторые особенности работы:
- **По умолчанию** элементы размещаются по строкам (слева направо, затем сверху вниз).
- **Если места для размещения элемента недостаточно,** сетка оставляет пробел и переходит на следующий трек.
- **При автоматическом размещении** когда некоторые элементы охватывают несколько треков, в сетке могут появиться незаполненные ячейки.
- **После заполнения явной сетки** (или если явной сетки нет) автоматическое размещение также приводит к генерации неявных дорожек сетки.

Свойства грид-контейнера

- **Свойства**
- Для управления автоматическим размещением элементов в CSS Grid Layout используется **свойство grid-auto-flow**. Оно определяет алгоритм автоматического размещения, указывая направление потока и стратегию заполнения пустот. Некоторые значения свойства:
- **row** — элементы размещаются по строкам, добавляя новые строки по мере необходимости. Значение по умолчанию.
- **column** — элементы размещаются по столбцам, добавляя новые столбцы по мере необходимости.
- **dense** — алгоритм использует «плотный» алгоритм упаковки, который пытается заполнить дыры в сетке, если позже появятся более мелкие элементы. Это может привести к тому, что элементы появятся не по порядку, но при этом заполнят отверстия, оставленные более крупными элементами.
- **Важно**: свойство grid-auto-flow не наследуется.

Свойства грид-контейнера

Единицы измерения

- **Функция minmax()** позволяет **задавать минимальный и максимальный размеры для строки или столбца**.
- Это полезно, когда нужно, чтобы элемент сохранял минимальный размер, но при этом мог увеличиваться при наличии свободного пространства
- **Создание гибких колонок**. Например:

```
Grid-template-columns: minmax(150px, 300 px);
```

- — колонка регулируется между 150px и 300px в зависимости от доступного пространства.

Свойства грид-контейнера: display

- Свободное пространство рассчитывается после того, как место отдано всем фиксированным размерам. К примеру, в коде ниже сначала будет создана колонка шириной 200 пикселей, а затем свободное пространство — ширина родителя минус 200 пикселей — будет поделено между остальными колонками. Каждая будет занимать ширину (100% - 200px) / 2:

```
.container {
  display: grid;
  grid-template-columns: 1fr 200px 1fr;
}
```

Свойства грид-контейнера

Отзывчивые сетки

- **auto-fill и auto-fit**
- В тех случаях, когда количество колонок или строк не известно для свойств grid-template-columns и grid-template-rows можно установить значения auto-fill или auto-fit в формате:

```
.container {
  grid-template-columns: repeat(/* auto-fill или auto-fit */, /* размер колонки */);
}
```

- Оба эти параметра (auto-fill и auto-fit) автоматически добавляют в грид-сетку из примера выше колонки. Но эти колонки ведут себя немного по-разному:
- auto-fill стремится заполнить колонками всё доступное пространство, а когда элементы заканчиваются, создаёт пустые колонки, равномерно распределяя доступную область между существующими и «виртуальными» колонками.
- auto-fit действует похожим образом, но, схлопывает пустые колонки и отдаёт больше места под заполненные.
- Аналогично можно использовать для создания рядов.

```
/* Минимальная ширина 150px, максимальная - любая (1fr) */
grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
```

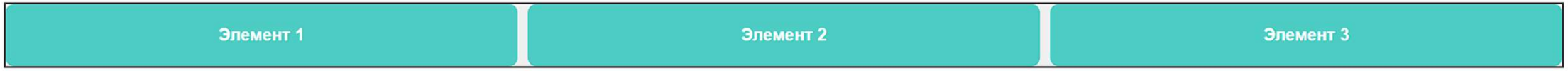
Пример с auto-fill

Обратите внимание: справа остались серые пустые ячейки. Красные блоки не растянулись.

```
/* Те же параметры: мин 150px, макс 1fr */
grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
```

Пример с auto-fit

Обратите внимание: пустых ячеек нет. Три элемента растянулись и заняли всю ширину.



Свойства грид-контейнера

- grid-auto-columns, grid-auto-rows Скопировать ссылку "grid-auto-columns, grid-auto-rows"
- Если элементов внутри грид-контейнера больше, чем может поместиться в заданные явно ряды и колонки, то для них создаются автоматические, неявные ряды и колонки. При помощи свойств grid-auto-columns и grid-auto-rows можно управлять размерами этих автоматических рядов и колонок.

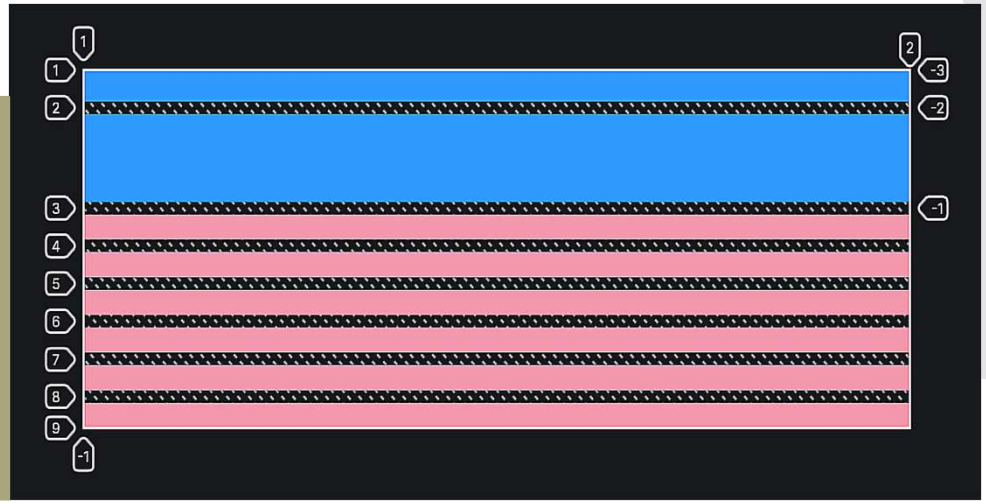
```
.container {
  display: grid;
  grid-template-rows: 50px 140px;
  grid-auto-rows: 40px;
  gap: 20px;
}
```

Свойства грид-контейнера

- В этом примере создаются два явных ряда размером 50 и 140 пикселей. Элементы, начиная с третьего, в эти два ряда не помещаются, и для них создаются автоматические ряды. При помощи свойства grid-auto-rows мы указываем, что автоматические ряды должны иметь размер 40 пикселей.

- Можно задавать больше одного значения для автоматических колонок или рядов. Тогда паттерн размера будет повторяться до тех пор, пока не кончатся грид-элементы.
- Важно указать при помощи grid-auto-flow: column, что элементы должны вставать в колонки, чтобы элементы без контента были видны.

```
.container {
  display: grid;
  grid-template-columns: 200px 500px;
  grid-auto-columns: 75px 100px 50px;
  grid-auto-flow: column;
  gap: 20px;
}
```



Свойства грид-контейнера

Автомаическое размещение

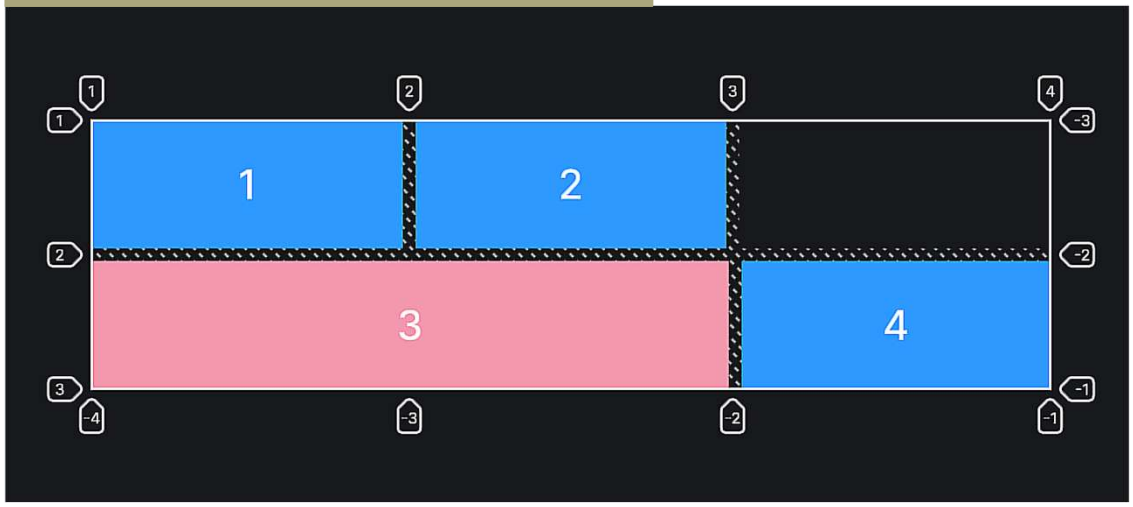
- Если грид-элементов больше, чем явно объявленных колонок или рядов, то они автоматически размещаются внутри родителя. А вот каким образом — в ряд или в колонку — можно указать при помощи свойства grid-auto-flow. По умолчанию значение у этого свойства row, лишние элементы будут выстраиваться в ряды в рамках явно заданных колонок.

- Возможные значения:

- row (значение по умолчанию) — автоматически размещаемые элементы выстраиваются в ряды.
- column — автоматически размещаемые элементы выстраиваются в колонки.
- dense — браузер старается заполнить дырки (пустые ячейки) в разметке, если размеры элементов позволяют. Можно сочетать с остальными значениями.
- Принципы работы этого свойства удобнее всего изучать на примере, когда есть большой блок, который не помещается в одну грид-ячейку.

Свойства грид-контейнера

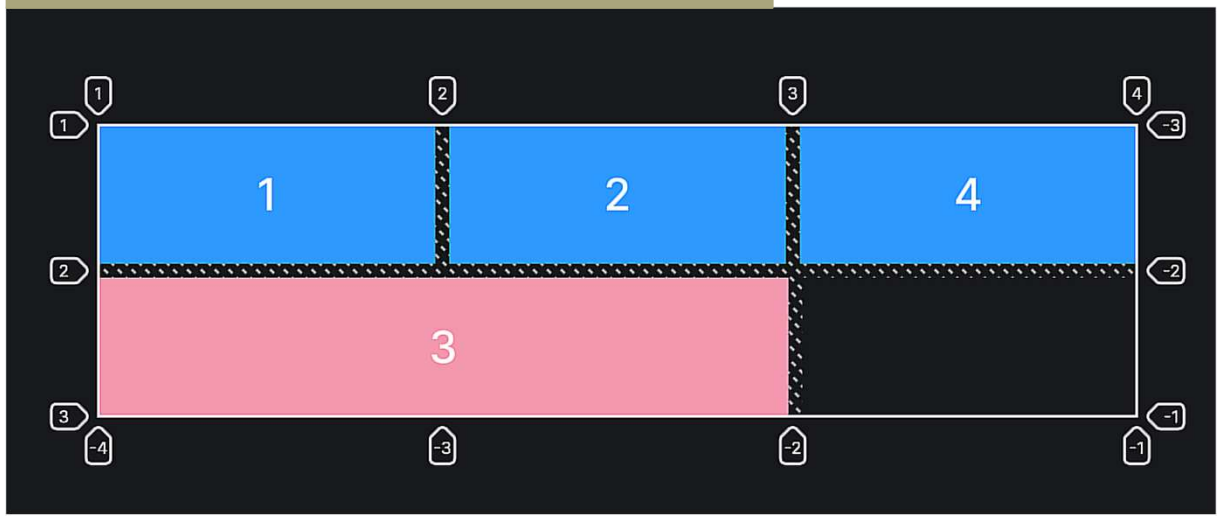
```
.container {
  display: grid;
  /* Три колонки */
  grid-template-columns: auto auto auto;
  /* Два ряда */
  grid-template-rows: auto auto;
  /* Автоматическое размещение в ряд */
  grid-auto-flow: row;
  /* Отступы между ячейками */
  gap: 20px;
}

.item3 {
  /* Занимает один ряд и
  растягивается на две колонки */
  grid-column: span 2;
}
```



Свойства грид-контейнера

- Как видите, третий элемент не поместился в последнюю ячейку первого ряда и был перенесён в следующий ряд. Следующий за ним четвёртый элемент встал в ближайшую доступную ячейку во втором ряду.
- Добавим к значению свойства grid-auto-flow слово dense:

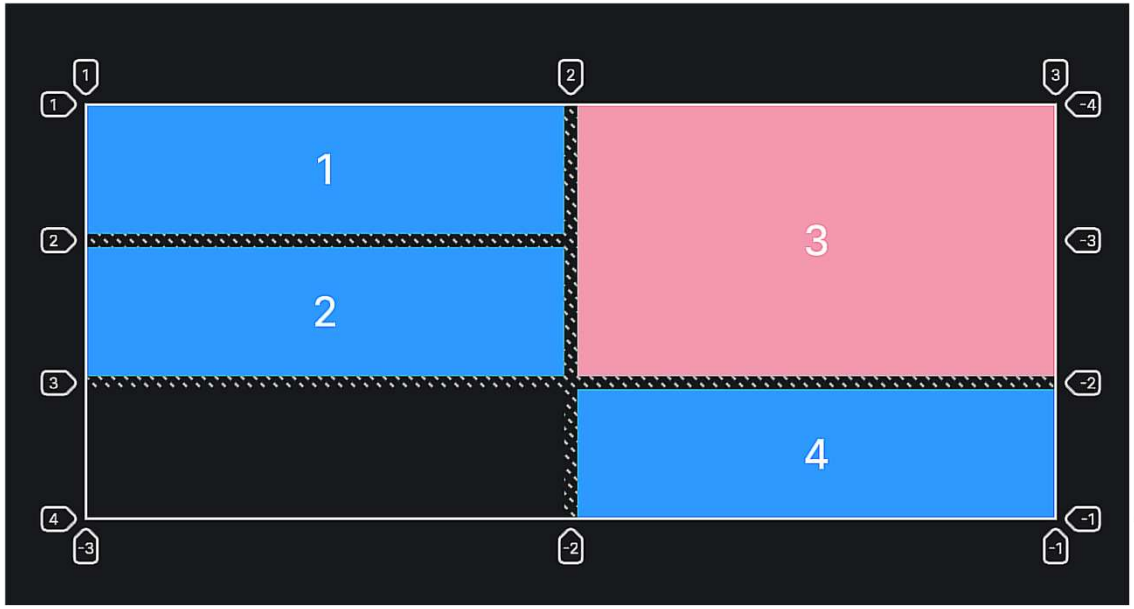
```
.container {
  /* Автоматическое размещение в ряд
*/
  grid-auto-flow: row dense;
}
```



Свойства грид-контейнера

- Теперь четвёртый элемент встал в ряд, но занял при этом пустую ячейку в первом ряду. Браузер старается закрыть все *дырки* в сетке, переставляя подходящие элементы на свободные места.
- Посмотрим пример со значением column:

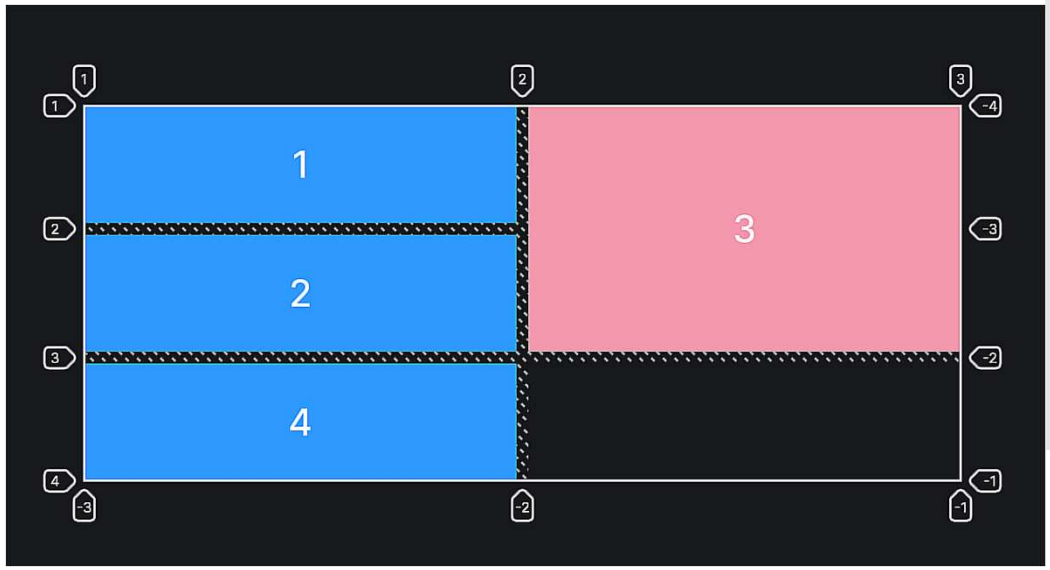
```
.container {
  grid-template-columns: auto auto;
  grid-template-rows: auto auto auto;
  /* Автоматическое размещение в
колонку */
  grid-auto-flow: column;
}

.item3 {
  grid-row: span 2;
}
```



Свойства грид-контейнера

- Видим аналогичную картину: третий элемент не поместился в последнюю ячейку первой колонки и встал во вторую колонку. Следующий за ним четвёртый элемент встал ниже во второй колонке.
- Добавим dense к текущему значению:

```
.container {
  /* Автоматическое размещение в ряд
*/
  grid-auto-flow: column dense;
}
```

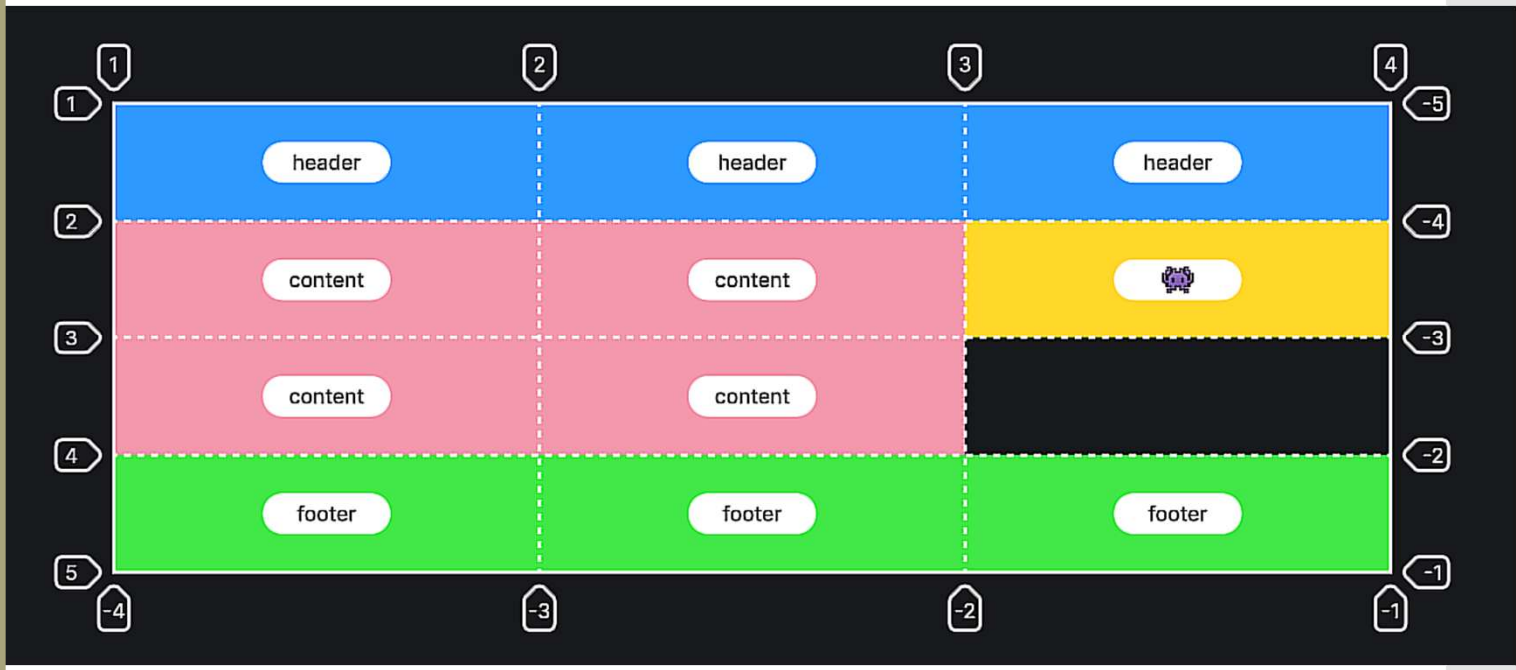


Свойства грид-контейнера

Шаблоны областей

- **grid-template-areas**
- Позволяет задать шаблон сетки расположения элементов внутри грид-контейнера. Имена областей задаются при помощи свойства grid-area. Текущее свойство grid-template-areas просто указывает, где должны располагаться эти грид-области.
- Возможные значения:
- none (значение по умолчанию) — области сетки не задано имя.
- . — означает пустую ячейку.
- название — название области, может быть абсолютно любым словом или даже эмодзи! 🤯
- Обратите внимание, что нужно называть каждую из ячеек. Например, если шапка или подвал нашего сайта будут занимать все три существующие колонки, то нужно будет трижды написать названия этих областей. Удобнее всего будет подписывать области в виде некой таблицы.

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 500px);
  grid-template-rows: repeat(4, 1fr);
  grid-template-areas:
    "header header header"
    "content content 👾"
    "content content ."
    "footer footer footer";
}

.item1 {
  grid-area: header;
}

.item2 {
  grid-area: content;
}

.item3 {
  grid-area: 👾;
}

.item4 {
  grid-area: footer;
}
```



Свойства грид-контейнера

- **grid-template**
- Свойство-шорткат для свойств grid-template-rows, grid-template-columns. Позволяет записать все значения в одну строку. Главное после этого не запутаться при чтении
- Можно прописать все колонки и ряды сразу, разделяя их слэшем /. Сперва идут ряды, а затем колонки, не перепутайте!

```
.container {
  display: grid;
  grid-template: repeat(4, 1fr) / repeat(3, 500px);
}
```

- В этом же свойстве можно задавать значение и для свойства grid-template-areas , но тогда код превращается в кашу и становится совершенно нечитабельным. Лучше всё же использовать это свойство отдельно:

```
.container {
  display: grid;
  grid-template:
    [row1-start] "header header header" 25px [row1-end]
    [row2-start] "footer footer footer" 25px [row2-end]
    / auto 50px auto;
}
```

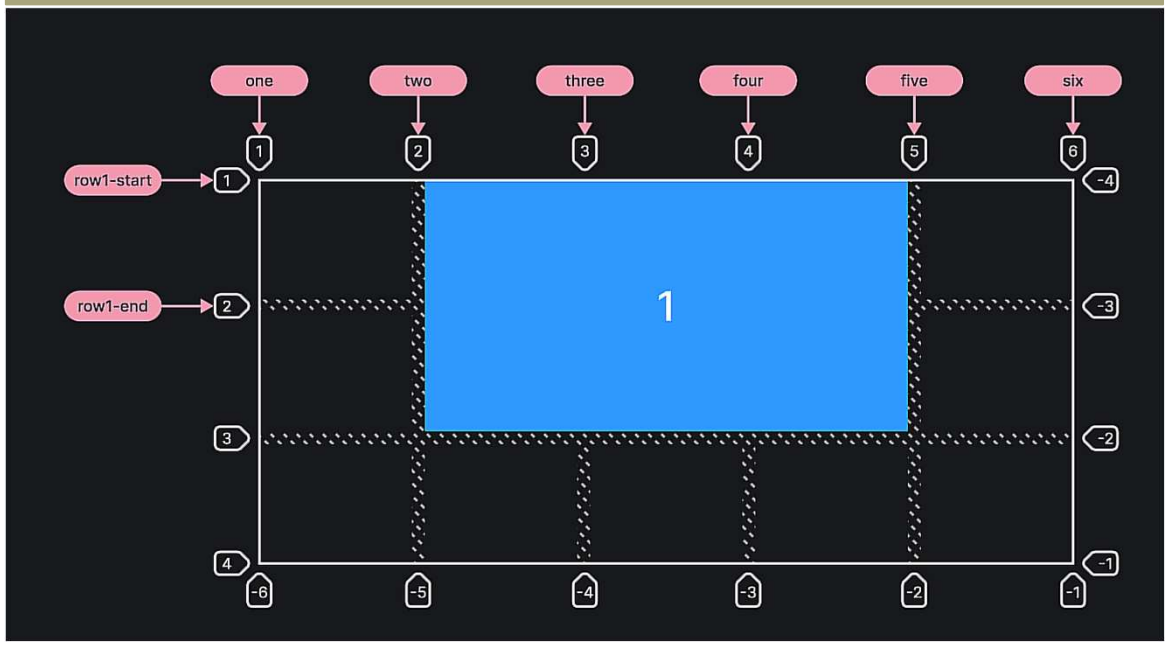
Свойства грид-элементов

Размещение элементов

- **grid-column-start, grid-column-end, grid-row-start, grid-row-end**
- Определяют положение элемента внутри грид-сетки при помощи указания на конкретные направляющие линии.
- Возможные значения:
- название или номер линии — может быть порядковым номером или названием конкретной линии.
- span число — элемент растянется на указанное количество ячеек.
- span имя — элемент будет растягиваться до следующей указанной линии.
- auto — означает автоматическое размещение, автоматический диапазон клеток или дефолтное растягивание элемента, равное одному.

Свойства грид-элементов

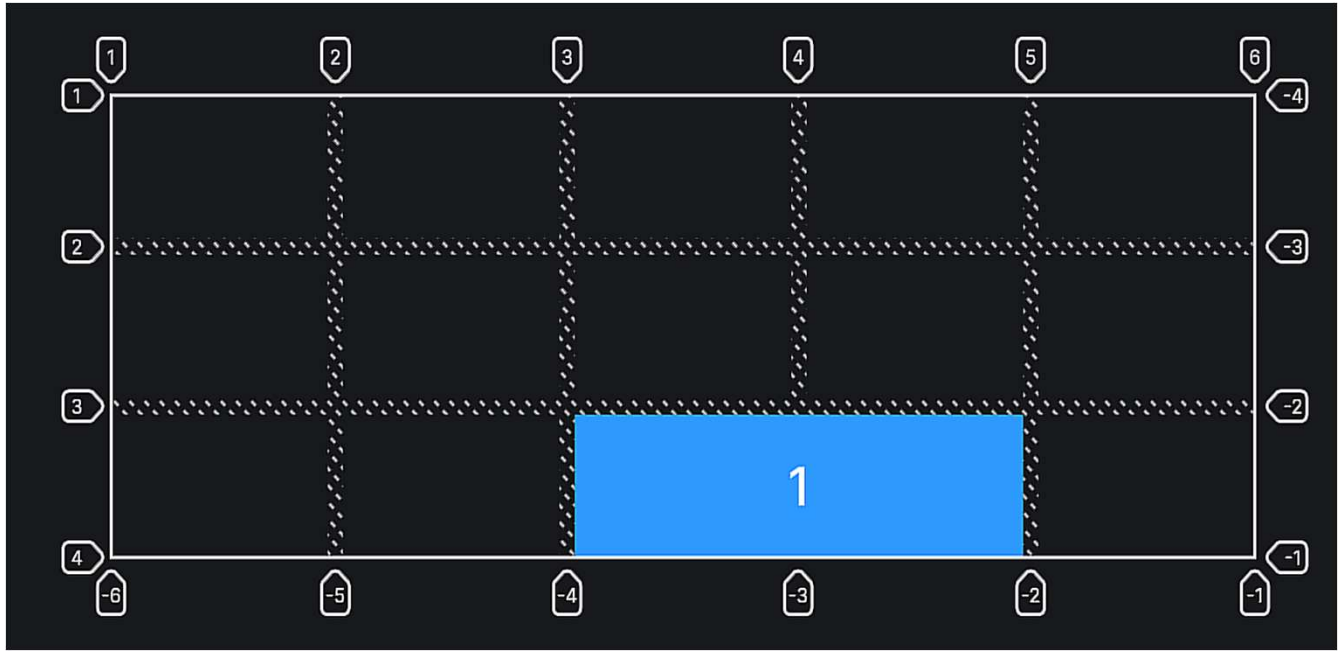
```
.container {
  display: grid;
  grid-template-columns: [one] 1fr [two] 1fr [three] 1fr [four] 1fr [five] 1fr [six];
  grid-template-rows: [row1-start] 1fr [row1-end] 1fr 1fr;
}

.item1 {
  grid-column-start: 2;
  grid-column-end: five;
  grid-row-start: row1-start;
  grid-row-end: 3;
}
```



Свойства грид-элементов

- **grid-column, grid-row**
- Свойства-шорткаты для grid-column-start, grid-column-end и grid-row-start, grid-row-end соответственно.
- Значения для *-start и *-end разделяются слэшем.
- Можно использовать ключевое слово span, буквально говорящее «растянись на столько-то». А на сколько, указывает стоящая за ним цифра.

```
.item1 {
  grid-column: 3 / span 2;
  grid-row: 3 / 4;
}
```



Свойства грид-элементов

- **grid-area**
- Двуличное свойство, которое может как указывать элементу, какую из именованных областей ему нужно занять, так и служить шорткатом для одновременного указания значений для четырёх свойств: grid-column-start, grid-column-end, grid-row-start и grid-row-end.
- Пример с указанием на именованную область:

```
.item1 {
  /* Займёт область content внутри грид-сетки */
  grid-area: content;
}
```

- А теперь пример с указанием всех четырёх значений в строку. При таком указании значений есть скользкое место: значения указываются в таком порядке: row-start / column-start / row-end / column-end, то есть сначала указываем оба начала, а потом оба конца.

```
.item2 {
  grid-area: 1 / col4-start / last-line / 6;
}
```

Небольшой тест

- **Вопрос 1 (1 правильный из 4)**
- Что такое CSS Grid Layout?
- a) Технология для работы только с горизонтальной раскладкой элементов.
- b) Удобная технология для раскладки элементов на веб-страницах, позволяющая работать с горизонталью и вертикалью одновременно.
- c) Спецификация для создания таблиц в HTML.
- d) Инструмент для анимации элементов на странице.

Небольшой тест

- **Вопрос 2 (Сопоставить пары, 6 пар)**
- Сопоставьте термины с их определениями:
- Грид-контейнер
- Грид-элемент
- Грид-линия
- Грид-ячейка
- Грид-полоса
- Грид-область

- A. Пространство между двумя соседними грид-линиями (ряд или колонка).
- B. Родительский элемент с display: grid.
- C. Область, ограниченная четырьмя грид-линиями, состоящая из ячеек.
- D. Разделительная линия, формирующая структуру грида.
- E. Пространство между соседними грид-линиями (единица сетки).
- F. Дочерний элемент грид-контейнера.

Небольшой тест

- **Вопрос 3 (Да/Нет)**
- Грид-контейнер с display: inline-grid ведет себя снаружи как строчный элемент?
- **Вопрос 4 (1 правильный из 5)**
- Какое свойство задает размеры и количество колонок в грид-раскладке?
- a) grid-auto-columns
- b) grid-template-rows
- c) grid-template-columns
- d) grid-auto-flow
- e) grid-area

Небольшой тест

- **Вопрос 5 (Выбрать несколько правильных)**
- Какие способы можно использовать для создания повторяющихся колонок или рядов?
- a) Функция repeat()
- b) Единица fr
- c) Ключевое слово auto-fill
- d) Свойство display: grid
- e) Функция minmax()
- **Вопрос 6 (Короткий ответ)**
- Как называется единица измерения, отвечающая за свободное пространство внутри грид-контейнера?

Небольшой тест

- **Вопрос 7 (1 правильный из 4)**
- Что означает значение auto для размера ряда или колонки?
- a) Размер фиксирован в 100px.
- b) Размер подстраивается под самый большой элемент и может растянуться до конца контейнера.
- c) Создает пустые колонки для заполнения пространства.
- d) Указывает на автоматическое размещение в колонки.
- **Вопрос 8 (Да/Нет)**
- При автоматическом размещении элементы по умолчанию размещаются по строкам (слева направо, сверху вниз)?

Небольшой тест

- **Вопрос 9 (Выбрать несколько правильных)**
- Какие значения может принимать свойство grid-auto-flow?
- a) row
- b) column
- c) dense
- d) auto
- **Вопрос 10 (1 правильный из 4)**
- В чем разница между auto-fill и auto-fit?
- a) auto-fill создает пустые колонки, а auto-fit схлопывает их и отдает место заполненным.
- b) auto-fit используется только для рядов, auto-fill — для колонок.
- c) auto-fill работает с fr, auto-fit — с minmax().
- d) Нет разницы, это синонимы.

Небольшой тест

- **Вопрос 11 (Короткий ответ)**
- Какое свойство управляет размерами автоматических (неявных) рядов?
- **Вопрос 12 (Сопоставить пары, 4 пары)**
- Сопоставьте свойства грид-элементов с их описаниями:
- grid-column-start
- grid-row-end
- grid-area
- grid-column

- A. Шорткат для grid-column-start и grid-column-end.
- B. Определяет конец ряда для элемента.
- C. Может указывать на именованную область или быть шорткатом для четырех свойств (row-start / column-start / row-end / column-end).
- D. Определяет начало колонки для элемента.

Небольшой тестик

- **Вопрос 13 (Да/Нет)**
- Свойство grid-template-areas позволяет задавать шаблон сетки с использованием имен областей, где "." обозначает пустую ячейку?
- **Вопрос 14 (1 правильный из 4)**
- Что делает функция minmax() в grid-template-columns?
- a) Задает только минимальный размер.
- b) Задает минимальный и максимальный размеры для строки или столбца.
- c) Повторяет паттерн колонок.
- d) Указывает на автоматическое размещение.

Небольшой тестик

- **Вопрос 15 (Выбрать несколько правильных из 5)**
- Какие свойства являются шорткатами? (Выберите все подходящие)
- a) grid-template
- b) grid-column
- c) grid-auto-flow
- d) grid-row
- e) grid-auto-rows

А теперь проверь себя

- **Правильные ответы**

1. b) Удобная технология для раскладки элементов на веб-страницах, позволяющая работать с горизонталью и вертикалью одновременно.
2. 1-B, 2-F, 3-D, 4-E, 5-A, 6-C.
3. Да.
4. c) grid-template-columns.
5. a) Функция repeat(), c) Ключевое слово auto-fill, e) Функция minmax().
6. fr.
7. b) Размер подстраивается под самый большой элемент и может растянуться до конца контейнера.
8. Да.
9. a) row, b) column, c) dense.
10. a) auto-fill создает пустые колонки, а auto-fit схлопывает их и отдает место заполненным.
11. grid-auto-rows.
12. 1-D, 2-B, 3-C, 4-A.
13. Да.
14. b) Задает минимальный и максимальный размеры для строки или столбца.
15. a) grid-template, b) grid-column, d) grid-row.

Спасибо за внимание!